

Universidad Privada Domingo Savio
Facultad de Ingeniería
Ingeniería en Sistemas



Configuración de Servidor WEB

Joaquin Alessandro Felipez Rojas
Nicolas Sebastian Reguerin Meneses
David Willy Cruz Huanca

Asignatura: Taller de Sistemas Operativos II

Docente: Jose Abraham Mita Huanca

Fecha de entrega: 17 de septiembre de 2026

Equipo: Grupo 1 Sudoers

Cochabamba–Bolivia

2026

Índice

CAPÍTULO I

INTRODUCCIÓN 1

1.1 Introducción 1

CAPÍTULO II

OBJETIVOS, ALCANCE Y LÍMITES 2

2.1 Objetivos del trabajo 2

2.1.1 Objetivo general 2

2.1.2 Objetivos específicos 2

2.2 Alcance y Límites 2

2.2.1 Alcance 2

2.2.2 Límites 3

2.2.3 Entorno de trabajo 4

CAPÍTULO III

DESARROLLO DEL TRABAJO 5

3.1 Desarrollo del trabajo 5

3.1.1 Arquitectura del servicio 5

3.1.2 Instalación de paquetes en Debian 6

3.1.3 Generación del certificado SSL/TLS con OpenSSL 8

3.1.4 Configuración TLS global y parámetros de sesión 9

3.1.5 Soporte WebSocket para aplicaciones web interactivas 10

3.1.6 Configuración de Virtual Hosts y Proxies Inversos 11

3.1.6.1 Proxy inverso para CUPS (Servidor de Impresión) 11

3.1.6.2 Proxy inverso para Filebrowser (Servidor de Archivos Web) 12

3.1.6.3 Proxy inverso para Portainer (Panel de Administración) 13

3.1.6.4 Proxy inverso para Zabbix (Monitoreo de Infraestructura) 14

3.1.6.5 Cabeceras HTTP de proxy compartidas 15

3.1.7 Virtual host del sitio principal y servidor por defecto 15

3.1.8 Gestión del servicio con systemd y control de firewall 17

3.1.9 Pruebas de acceso y verificación 18

3.1.9.1 Verificación del arranque y puertos del servicio 18

3.1.9.2 Prueba del sitio institucional principal	19
3.1.9.3 Prueba de los proxies inversos	20
3.1.9.4 Verificación del certificado SSL/TLS	21
CONCLUSIONES	23
4. Conclusiones	23
4.1 Resultados obtenidos	23
4.2 Dificultades encontradas	24
4.3 Importancia del servidor web en la administración de redes	24
4.4 Aprendizajes adquiridos	25
REFERENCIAS	26

Índice de tablas

TABLA 1: Virtual hosts configurados en Nginx	6
TABLA 2: Puertos de red del servidor web	8
TABLA 3: Características del certificado SSL auto-firmado	9
TABLA 4: Directivas TLS globales de Nginx	10
TABLA 5: Cabeceras HTTP de los proxies inversos	15
TABLA 6: Comandos de administración de Nginx con <code>systemd</code>	18

Índice de figuras

FIGURA 1: Arquitectura del servidor web: Nginx como proxy inverso y servidor de contenido en Debian	5
FIGURA 2: Estado activo del servicio Nginx en Debian	19
FIGURA 3: Sitio web del proyecto accedido desde un navegador	20
FIGURA 4: Acceso a servicios internos a través del proxy inverso	21
FIGURA 5: Verificación del certificado SSL del servidor web	21

CAPÍTULO I

INTRODUCCIÓN

1.1 Introducción

En toda infraestructura de red institucional, el servidor web constituye la puerta de entrada a los recursos digitales de una organización. A través de los protocolos HTTP y HTTPS, un servidor web atiende las solicitudes de los navegadores de los usuarios, devolviendo páginas, archivos y servicios de forma segura Tanenbaum y Wetherall (2017).

En entornos empresariales, el servidor web frecuentemente cumple funciones adicionales como proxy inverso —redirigiendo las solicitudes de los clientes hacia servicios internos— y terminador TLS —cifrando las comunicaciones con certificados SSL/TLS Nginx, Inc. (2026). Esta capacidad permite centralizar el acceso a múltiples servicios bajo un único punto de entrada, simplificando la administración de la red y mejorando la seguridad al cifrar todas las comunicaciones entre los clientes y los servicios.

El presente informe documenta la configuración de un servidor web dentro del proyecto integrador del curso de Taller de Sistemas Operativos II, desarrollado por el equipo *Grupo 1 Sudoers* en el dominio `sudoers.lan`. Se implementó Nginx, configurado como servidor de contenido estático, proxy inverso hacia los servicios internos del proyecto (servidor de impresión CUPS, servidor de archivos web, plataforma de gestión Portainer y sistema de monitoreo Zabbix) y terminador TLS con certificados auto-firmados.

El servicio se despliega de manera nativa sobre un servidor Debian GNU/Linux 12 (Bookworm) mediante el gestor de paquetes APT y administrado por `systemd`, integrándose con el servidor DNS (BIND9) del proyecto para la resolución de nombres dentro del dominio `sudoers.lan`.

Este capítulo introduce el contexto general del taller y presenta la importancia del servidor web en la administración de sistemas informáticos. Los capítulos siguientes desarrollan los objetivos planteados, el alcance y los límites del trabajo, el proceso de configuración del servicio y las conclusiones obtenidas.

CAPÍTULO II

OBJETIVOS, ALCANCE Y LÍMITES

2.1 Objetivos del trabajo

2.1.1 Objetivo general

Comprender el funcionamiento de un servidor web (Nginx), así como su configuración e integración con los servicios de resolución de nombres (DNS) y los demás servicios de la infraestructura de red del proyecto `sudoers.lan`, en un entorno administrado con Debian.

2.1.2 Objetivos específicos

- Instalar y configurar de forma nativa el servidor web Nginx sobre Debian para alojar el sitio institucional del proyecto y servir como proxy inverso hacia los servicios internos.
- Implementar HTTPS con certificados SSL/TLS auto-firmados para la protección de las comunicaciones web.
- Configurar virtual hosts para múltiples subdominios del proyecto (`web.sudoers.lan`, `print.sudoers.lan`, `files.sudoers.lan`, `portainer.sudoers.lan`, `zabbix.sudoers.lan`).
- Configurar proxies inversos hacia los servicios internos del proyecto (CUPS, File-browser, Portainer y Zabbix) con soporte para WebSocket.
- Realizar pruebas de acceso web desde navegadores y verificación de los proxies inversos.
- Documentar el proceso completo de instalación, configuración y prueba del servicio, incluyendo comandos, archivos de configuración y evidencias de cada paso.

2.2 Alcance y Límites

2.2.1 Alcance

El presente informe documenta la configuración del servidor web del proyecto. Las actividades realizadas incluyen:

- Instalación y configuración nativa de Nginx sobre Debian GNU/Linux mediante el gestor de paquetes APT.
- Configuración de virtual hosts para el sitio principal del proyecto (`web.sudoers.lan`, `www.sudoers.lan`, `sudoers.lan`).
- Configuración de proxy inverso hacia los servicios internos:
 - CUPS (impresión): `print.sudoers.lan`
 - Filebrowser (archivos): `files.sudoers.lan`
 - Portainer (panel de administración): `portainer.sudoers.lan`
 - Zabbix (monitoreo): `zabbix.sudoers.lan`
- Generación de certificados SSL/TLS auto-firmados mediante la herramienta OpenSSL con directivas SAN (*Subject Alternative Name*).
- Configuración de redirección HTTP a HTTPS en todos los virtual hosts.
- Configuración del servidor por defecto (`default_server`) para capturar peticiones no mapeadas.
- Configuración de soporte WebSocket para Portainer.
- Despliegue de la landing page institucional como contenido estático en `/var/www/html`.
- Habilitación y control del demonio Nginx a través del sistema de inicio `systemd`.
- Pruebas de acceso desde navegadores y verificación de los proxies inversos.

2.2.2 Límites

Los siguientes aspectos no forman parte de este informe:

- Certificados SSL emitidos por una autoridad de certificación (CA) confiable; se utilizan certificados auto-firmados adecuados para un entorno de laboratorio.
- Integración con dominios externos o configuración de registros DNS públicos.
- Balanceo de carga o alta disponibilidad del servidor web.
- Configuración de WAF (*Web Application Firewall*) o reglas avanzadas de seguridad

en Nginx.

- Seguridad avanzada como protección contra ataques DDoS, rate limiting o listas de control de acceso por IP.

2.2.3 Entorno de trabajo

El entorno utilizado para la configuración consta de los siguientes componentes:

- Servidor: máquina física con Debian GNU/Linux 12 (Bookworm) donde se instalan y configuran los servicios de forma nativa.
- Despliegue: administración directa mediante paquetes oficiales de Debian y orquestación de servicios a través de `systemd`.
- Red interna: red administrativa 192.168.0.0/24 (WLO1) y red de usuarios 192.168.20.0/24 (ENO1).
- Servicio: Nginx (Debian GNU/Linux 12 Bookworm).
- Resolución de nombres: servidor DNS propio (BIND9) que resuelve el dominio `sudoers.lan` para todos los servicios.
- Clientes de prueba: máquinas Linux y Windows conectadas a la red local, navegadores web para verificación de los sitios y proxies inversos.
- Usuarios: `nicolas`, `joaquin` y `david`, integrantes del equipo *Grupo 1 Sudoers*.

CAPÍTULO III

DESARROLLO DEL TRABAJO

3.1 Desarrollo del trabajo

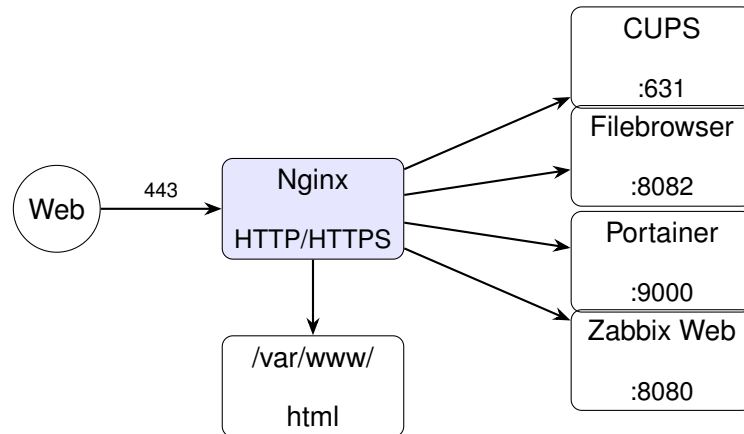
Este capítulo documenta el proceso de instalación, configuración y verificación del servidor web del proyecto sobre el sistema operativo Debian GNU/Linux 12 (Bookworm). Se detalla desde la instalación de los paquetes mediante APT hasta la puesta en marcha del demonio con `systemd`, la configuración de virtual hosts, la terminación TLS y las pruebas de conectividad y acceso desde los clientes de la red.

3.1.1 Arquitectura del servicio

El servidor web se implementó utilizando Nginx instalado de manera nativa sobre Debian. Nginx cumple un doble rol estratégico en la infraestructura de la organización: en primer término, aloja el sitio web institucional estático del equipo sirviendo páginas HTML5, estilos y recursos gráficos; en segundo término, actúa como proxy inverso centralizado hacia los diversos servicios internos que operan en la red local o en puertos internos del sistema (servidor de impresión CUPS, servicio web de archivos Filebrowser, panel de administración Portainer y panel de monitoreo Zabbix).

Todo el tráfico web entrante se encuentra protegido mediante el protocolo HTTPS a través de certificados TLS auto-firmados con soporte para nombres alternativos (*Subject Alternative Name*, SAN) en formato comodín (*wildcard*). La figura 1 ilustra la arquitectura general del servidor web implementado.

FIGURA 1: Arquitectura del servidor web: Nginx como proxy inverso y servidor de contenido en Debian



FUENTE: Elaboración propia.

La tabla 1 resume los virtual hosts configurados en Nginx, sus nombres de dominio asociados, sus destinos y la función que cumplen en la infraestructura.

TABLA 1: Virtual hosts configurados en Nginx

Subdominio	Destino	Función
print.sudoers.lan	http://127.0.0.1:631	Proxy inverso a CUPS (impresión)
files.sudoers.lan	http://127.0.0.1:8082	Proxy inverso a Filebrowser (archivos)
portainer.sudoers.lan	http://127.0.0.1:9000	Proxy inverso a Portainer (panel de gestión)
zabbix.sudoers.lan	http://127.0.0.1:8080	Proxy inverso a Zabbix (monitoreo)
web.sudoers.lan	/var/www/html	Sitio institucional del proyecto
www.sudoers.lan	/var/www/html	Alias del sitio institucional
sudoers.lan	/var/www/html	Alias del dominio principal
_ (default)	Redirección a HTTPS	Acceso por IP o dominios no declarados

FUENTE: Elaboración propia.

3.1.2 Instalación de paquetes en Debian

Todos los componentes necesarios para el funcionamiento del servidor web se instalaron directamente desde los repositorios oficiales de Debian GNU/Linux 12 utilizando el administrador de paquetes APT (*Advanced Package Tool*).

El comando ejecutado para actualizar el índice de repositorios e instalar Nginx junto con la biblioteca criptográfica OpenSSL fue el siguiente:

```
$ sudo apt update
$ sudo apt install -y nginx openssl
```

La descripción y justificación de cada paquete instalado se detalla a continuación:

- **nginx**: servidor web de arquitectura orientada a eventos altamente escalable, utilizado ampliamente como servidor HTTP y proxy inverso debido a su bajo consumo de recursos y alto rendimiento bajo cargas concurrentes.
- **openssl**: conjunto de herramientas y librerías criptográficas de código abierto indispensables para la generación de claves privadas, solicitudes de firma de certificados (CSR) y certificados digitales X.509 necesarios para la habilitación del protocolo HTTPS.

Tras la instalación en Debian, los archivos de configuración y directorios del servicio quedan organizados bajo el estándar del sistema de archivos FHS (*Filesystem Hierarchy Standard*):

- `/etc/nginx/nginx.conf`: archivo principal de directivas globales del servidor.
- `/etc/nginx/sites-available/`: directorio que alberga los archivos de configuración de los virtual hosts disponibles en el servidor.
- `/etc/nginx/sites-enabled/`: directorio que contiene los enlaces simbólicos hacia los archivos de `sites-available/` que se encuentran activos.
- `/var/www/html/`: directorio raíz predeterminado donde se alojan los archivos HTML, CSS e imágenes del sitio web.
- `/etc/ssl/certs/` y `/etc/ssl/private/`: rutas estándar donde se almacenan los certificados públicos y las claves privadas respectivamente.

La tabla 2 resume los puertos de red asignados al servicio web en el servidor Debian.

TABLA 2: Puertos de red del servidor web

Puerto	Protocolo	Servicio
80	TCP	HTTP (redirige permanentemente a HTTPS)
443	TCP	HTTPS (tráfico cifrado TLS para sitio y proxies)

FUENTE: Elaboración propia.

3.1.3 Generación del certificado SSL/TLS con OpenSSL

Para garantizar la confidencialidad e integridad del tráfico de red entre los clientes y el servidor, se generó un certificado SSL auto-firmado de 2048 bits mediante la herramienta `openssl`. El certificado incluye el nombre base del dominio (`sudoers.lan`) y la extensión comodín (`*.sudoers.lan`) en el parámetro `subjectAltName`, permitiendo proteger simultáneamente el sitio web principal y todos los subdominios de proxy inverso.

Los comandos ejecutados en el servidor Debian para la preparación de directorios, generación del par de claves y asignación de permisos de seguridad fueron:

```
# Crear directorios de certificados con permisos restrictivos
$ sudo mkdir -p /etc/ssl/certs /etc/ssl/private

# Generar clave privada y certificado auto-firmado X.509
$ sudo openssl req -new -x509 -days 3650 -nodes \
    -out /etc/ssl/certs/web.pem \
    -keyout /etc/ssl/private/web.key \
    -subj "/CN=sudoers.lan/O=TSO-II/C=AR" \
    -addext "subjectAltName = DNS:sudoers.lan, DNS:*.sudoers.lan"

# Establecer permisos de seguridad en la clave privada y certificado
$ sudo chmod 600 /etc/ssl/private/web.key
$ sudo chmod 644 /etc/ssl/certs/web.pem
```

El significado de los parámetros empleados en la invocación de `openssl` comprende:

- `-new -x509`: instruye a la herramienta para generar directamente un certificado auto-firmado de acuerdo con la norma ITU-T X.509 en lugar de una solicitud de firma

(CSR).

- `-days 3650`: define un periodo de vigencia de 10 años para el certificado, idóneo para entornos de laboratorio y servidores de red interna.
- `-nodes`: (*no DES*) especifica que la clave privada generada no debe cifrarse con contraseña, permitiendo que el demonio Nginx se inicie de manera desatendida durante el arranque del sistema operativo.
- `-subj`: establece los campos del nombre distinguido (DN), asignando `CN=sudoers.lan` como nombre común y la organización TSO-II.
- `-addext`: agrega extensiones X.509 v3, específicamente `subjectAltName`, que define los nombres de dominio cubiertos por el certificado.

La tabla 3 describe las características técnicas del certificado TLS emitido.

TABLA 3: Características del certificado SSL auto-firmado

Parámetro	Valor
Tipo	X.509 v3, auto-firmado
Algoritmo	RSA 2048 bits
Validez	3650 días (10 años)
CN (<i>Common Name</i>)	<code>sudoers.lan</code>
Organización	TSO-II
País	AR
<code>subjectAltName</code>	<code>DNS:sudoers.lan, DNS:*.sudoers.lan</code>
Ubicación pública	<code>/etc/ssl/certs/web.pem</code>
Ubicación clave privada	<code>/etc/ssl/private/web.key</code> (modo 0600)

FUENTE: Elaboración propia.

3.1.4 Configuración TLS global y parámetros de sesión

Para asegurar que todas las comunicaciones HTTPS cumplan con los estándares de seguridad contemporáneos, se establecieron parámetros TLS globales dentro del bloque `http` de Nginx (`/etc/nginx/nginx.conf` o archivo incluido en `/etc/nginx/conf.d/tls.conf`):

```
# Protocolos criptograficos admitidos
ssl_protocols TLSv1.2 TLSv1.3;
ssl_prefer_server_ciphers off;
```

```
# Gestion de cache de sesiones TLS
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1d;
```

La tabla 4 detalla el propósito de cada directiva TLS aplicada.

TABLA 4: Directivas TLS globales de Nginx

Directiva	Justificación
ssl_protocols	Limita las conexiones a TLS 1.2 y 1.3, bloqueando versiones inseguras como SSLv3, TLS 1.0 y 1.1
ssl_prefer_server_ciphers off	Respetar la selección de suites de cifrado modernas del cliente según las mejores prácticas
ssl_session_cache	Asigna un área compartida de memoria de 10 MB para reutilizar parámetros de sesión TLS
ssl_session_timeout	Fija la expiración de las sesiones en 24 horas, reduciendo la sobrecarga de cálculo de clave

FUENTE: Elaboración propia.

3.1.5 Soporte WebSocket para aplicaciones web interactivas

Determinadas interfaces de administración, como el panel de gestión Portainer, requieren el mantenimiento de túneles bidireccionales en tiempo real mediante el protocolo WebSocket. Para evitar que el proxy inverso cierre o degrade estas conexiones, se configuró el bloque map a nivel HTTP en `nginx.conf`:

```
map $http_upgrade $connection_upgrade {
    default upgrade;
    '' close;
}
```

Este mecanismo evalúa dinámicamente la presencia de la cabecera HTTP Upgrade enviada por el navegador: si la solicitud solicita una conversión a WebSocket, la variable `$connection_upgrade` adopta el valor `upgrade`, permitiendo que Nginx conmute el canal de comunicación de forma transparente.

3.1.6 Configuración de Virtual Hosts y Proxies Inversos

La configuración integral de los sitios virtuales y proxies inversos se plasmó en el archivo `/etc/nginx/sites-available/sudoers.conf`, habilitado en el servidor mediante un enlace simbólico en `/etc/nginx/sites-enabled/`. A continuación se documenta la estructura de cada virtual host configurado.

3.1.6.1 Proxy inverso para CUPS (Servidor de Impresión)

El servidor de impresión CUPS atiende en el puerto local 631. Para exponer su interfaz gráfica en el subdominio `print.sudoers.lan` bajo HTTPS, se configuró la redirección obligatoria desde el puerto 80 y la terminación TLS en el puerto 443:

```
# Redirección de tráfico plano a HTTPS
server {
    listen 80;
    server_name print.sudoers.lan;
    return 301 https://$host$request_uri;
}

# Proxy inverso con terminación TLS
server {
    listen 443 ssl;
    server_name print.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        proxy_pass http://127.0.0.1:631;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
```

```
    proxy_set_header X-Forwarded-Proto $scheme;
}
}
```

3.1.6.2 Proxy inverso para Filebrowser (Servidor de Archivos Web)

El servidor de archivos Filebrowser ofrece una interfaz web de gestión y transferencia de ficheros. Dado que los usuarios requieren cargar archivos de gran tamaño, se amplió la directiva `client_max_body_size` a 500 MB:

```
server {
    listen 80;
    server_name files.sudoers.lan;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name files.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    client_max_body_size 500M;

    location / {
        proxy_pass http://127.0.0.1:8082;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

3.1.6.3 Proxy inverso para Portainer (Panel de Administración)

Para el panel de administración Portainer (puerto 9000), se implementó el paso de parámetros para WebSocket junto con temporizadores de lectura y envío extendidos a 3600 segundos (1 hora), garantizando sesiones interactivas estables sin desconexiones prematuras:

```
server {
    listen 80;

    server_name portainer.sudoers.lan;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;

    server_name portainer.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        proxy_pass http://127.0.0.1:9000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Parametros para tunelizacion WebSocket
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection $connection_upgrade;
```

```
    proxy_read_timeout 3600;
    proxy_send_timeout 3600;
}
}
```

3.1.6.4 Proxy inverso para Zabbix (Monitoreo de Infraestructura)

El panel de supervisión y monitoreo del servidor se configuró bajo el subdominio `zabbix.sudoers.lan`, canalizando el tráfico al servicio web de Zabbix:

```
server {
    listen 80;
    server_name zabbix.sudoers.lan;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl;
    server_name zabbix.sudoers.lan;

    ssl_certificate      /etc/ssl/certs/web.pem;
    ssl_certificate_key  /etc/ssl/private/web.key;

    location / {
        proxy_pass http://127.0.0.1:8080;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

3.1.6.5 Cabeceras HTTP de proxy compartidas

En todas las directivas de proxy inverso se remitieron cabeceras específicas para mantener el contexto del cliente original frente a los servicios de backend. La tabla 5 sintetiza la función de cada una.

TABLA 5: Cabeceras HTTP de los proxies inversos

Cabecera	Función
Host	Transmite el nombre de dominio original solicitado por el cliente
X-Real-IP	Informa la dirección IP real del cliente remoto al backend
X-Forwarded-For	Preserva la lista de proxies intermediarios en la petición
X-Forwarded-Proto	Indica el protocolo de origen utilizado por el cliente (https)

FUENTE: Elaboración propia.

3.1.7 Virtual host del sitio principal y servidor por defecto

El sitio web institucional del proyecto se publica como contenido estático alojado en el directorio `/var/www/html/`. Responde a las peticiones dirigidas a `web.sudoers.lan`, `www.sudoers.lan` y `sudoers.lan`:

```
server {  
    listen 80;  
    server_name web.sudoers.lan www.sudoers.lan sudoers.lan;  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl;  
    server_name web.sudoers.lan www.sudoers.lan sudoers.lan;  
  
    ssl_certificate      /etc/ssl/certs/web.pem;  
    ssl_certificate_key  /etc/ssl/private/web.key;
```

```
root /var/www/html;  
index index.html;  
  
location / {  
    try_files $uri $uri/ =404;  
}  
}
```

Para el despliegue del sitio, los archivos HTML, CSS y recursos gráficos se copiaron directamente al directorio raíz del servidor web en Debian, asignando la propiedad al usuario y grupo del sistema `www-data`:

```
$ sudo cp -r services/web/public/* /var/www/html/  
$ sudo chown -R www-data:www-data /var/www/html  
$ sudo chmod -R 755 /var/www/html
```

Adicionalmente, se implementó el bloque `default_server` para gestionar de forma controlada cualquier conexión que ingrese directamente mediante la dirección IP del servidor o a través de dominios no contemplados:

```
server {  
    listen 80 default_server;  
    server_name _;  
    return 301 https://$host$request_uri;  
}  
  
server {  
    listen 443 ssl default_server;  
    server_name _;  
  
    ssl_certificate      /etc/ssl/certs/web.pem;  
    ssl_certificate_key  /etc/ssl/private/web.key;
```

```
    root /var/www/html;

    index index.html;


    location / {
        try_files $uri $uri/ =404;
    }
}
```

3.1.8 Gestión del servicio con systemd y control de firewall

La habilitación, supervisión y administración del servicio Nginx en Debian se efectúa a través del sistema de inicio y control de procesos `systemd`.

Antes de iniciar o recargar el servicio, se ejecutó la comprobación sintáctica de los archivos de configuración mediante el comando de validación de Nginx:

```
$ sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Una vez verificada la corrección sintáctica, se habilitó y arrancó el servicio del sistema:

```
# Habilitar el servicio para arranque automatico al encender el equipo
$ sudo systemctl enable nginx

# Iniciar o reiniciar el demonio
$ sudo systemctl restart nginx

# Inspeccionar el estado operativo del servicio
$ sudo systemctl status nginx
```

La tabla 6 detalla los comandos de administración del servicio web en Debian.

TABLA 6: Comandos de administración de Nginx con `systemd`

Comando	Acción
<code>systemctl enable nginx</code>	Habilita el inicio automático del servicio durante el arranque
<code>systemctl start nginx</code>	Inicia el demonio del servidor web
<code>systemctl stop nginx</code>	Detiene la ejecución del servidor web
<code>systemctl restart nginx</code>	Reinicia el proceso completamente
<code>systemctl reload nginx</code>	Recarga los archivos de configuración sin interrumpir conexiones
<code>systemctl status nginx</code>	Muestra el estado activo, PID y registros de ejecución recientes

FUENTE: Elaboración propia.

Para permitir el acceso a través del cortafuegos de Debian (`nftables` o `ufw`), se autorizó el tráfico entrante en los puertos 80 y 443:

```
$ sudo ufw allow 80/tcp comment 'Nginx HTTP'
$ sudo ufw allow 443/tcp comment 'Nginx HTTPS'
$ sudo ufw reload
```

3.1.9 Pruebas de acceso y verificación

3.1.9.1 Verificación del arranque y puertos del servicio

Se comprobó que el servicio Nginx se encuentra en estado activo (*running*) y escuchando correctamente en las interfaces de red sobre los puertos 80 y 443:

```
$ sudo systemctl status nginx
* nginx.service – A high performance web server and a reverse proxy
server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled)
   Active: active (running)

$ sudo ss -tulpn | grep nginx
tcp    LISTEN 0 511      0.0.0.0:80      0.0.0.0:*     users:(("nginx",pid
=1234,fd=6))
```

```
tcp    LISTEN 0 511      0.0.0.0:443      0.0.0.0:*        users : ( "nginx", pid
=1234, fd=7) )
```

FIGURA 2: Estado activo del servicio Nginx en Debian

```
david@sudoers:~$ docker logs tso-web
>>> Generando certificado SSL auto-firmado para sudoers.lan...
>>> Certificado generado: /etc/ssl/certs/web.pem
2026/09/16 13:27:08 [notice] 1#1: using the "epoll" event method
2026/09/16 13:27:08 [notice] 1#1: nginx/1.27.5
2026/09/16 13:27:08 [notice] 1#1: built by gcc 14.2.0 (Alpine 14.2.0)
2026/09/16 13:27:08 [notice] 1#1: OS: Linux 6.12.107+deb13-amd64
2026/09/16 13:27:08 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1024:524288
2026/09/16 13:27:08 [notice] 1#1: start worker processes
2026/09/16 13:27:08 [notice] 1#1: start worker process 10
2026/09/16 13:27:08 [notice] 1#1: start worker process 11
2026/09/16 13:27:08 [notice] 1#1: start worker process 12
2026/09/16 13:27:08 [notice] 1#1: start worker process 13
192.168.0.101 - - [16/Sep/2026:13:28:00 +0000] "GET / HTTP/1.1" 301 169 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:00 +0000] "GET / HTTP/1.1" 200 33933 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:09 +0000] "GET /assets/img/david-endavour.webp HTTP/1.1" 200 4446 "https://192.168.0.105/" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:09 +0000] "GET /assets/img/nicolas-cachyos.webp HTTP/1.1" 200 4820 "https://192.168.0.105/" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:09 +0000] "GET /assets/img/logo-sudoers.webp HTTP/1.1" 200 18750 "https://192.168.0.105/" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:09 +0000] "GET /assets/img/joaquin-debian.webp HTTP/1.1" 200 4918 "https://192.168.0.105/" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:10 +0000] "GET /assets/img/logo-sudoers.png HTTP/1.1" 200 234198 "https://192.168.0.105/" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
192.168.0.101 - - [16/Sep/2026:13:28:55 +0000] "HEAD / HTTP/1.1" 200 0 "-" "curl/8.14.1" "-"
192.168.0.101 - - [16/Sep/2026:13:29:40 +0000] "GET / HTTP/1.1" 200 33933 "-" "curl/8.14.1" "-"
192.168.0.101 - - [16/Sep/2026:13:30:25 +0000] "GET / HTTP/1.1" 200 33933 "-" "Mozilla/5.0 (X11; Linux x86_64; rv:155.0) Gecko/20100101 Firefox/155.0" "-"
```

FUENTE: Captura de pantalla mostrando la salida de `systemctl status nginx` y los puertos de escucha del servidor web en el sistema.

3.1.9.2 Prueba del sitio institucional principal

Desde una estación de trabajo de la red local, se verificó la redirección HTTP y la entrega del contenido en HTTPS:

```
# Verificacion de redireccion 301 desde HTTP a HTTPS
$ curl -I http://web.sudoers.lan
HTTP/1.1 301 Moved Permanently
Server: nginx
Location: https://web.sudoers.lan/

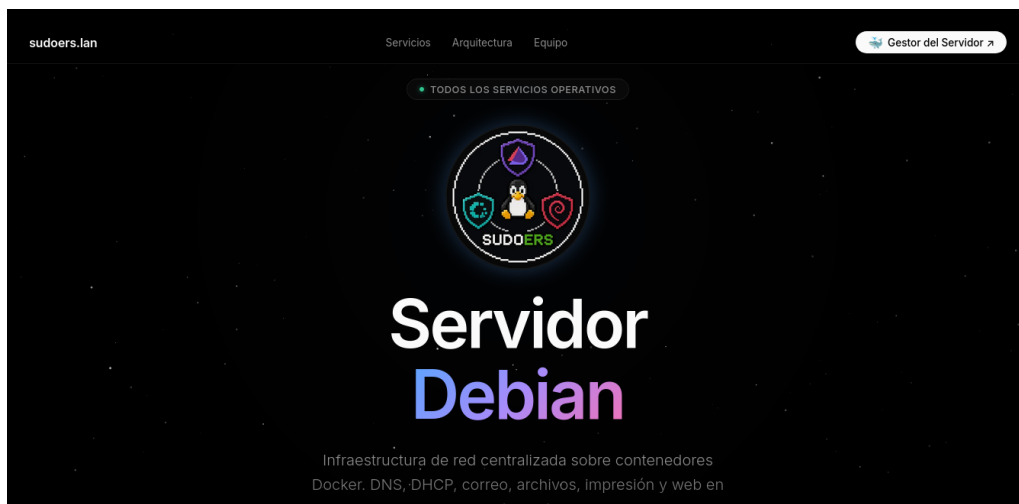
# Acceso seguro por HTTPS (se emplea -k por certificado auto-firmado)
$ curl -k https://web.sudoers.lan
<!DOCTYPE html>
<html lang="es">
<head>
```

```
<meta charset="UTF-8">

<title>Grupo 1 Sudoers – Infraestructura de Servidores</title>

...
```

FIGURA 3: Sitio web del proyecto accedido desde un navegador



FUENTE: Captura de pantalla del navegador web mostrando la landing page institucional `sudoers.lan` con certificado HTTPS activo.

3.1.9.3 Prueba de los proxies inversos

Se comprobó que las peticiones hacia cada uno de los subdominios son dirigidas satisfactoriamente hacia el servicio correspondiente:

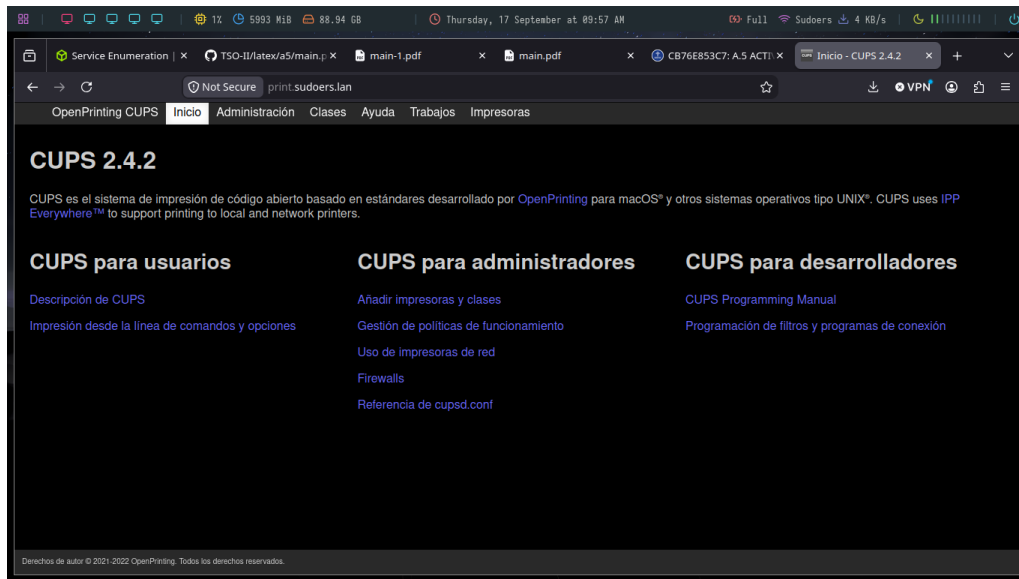
```
# Verificacion de proxy hacia CUPS (impresion)
$ curl -k -I https://print.sudoers.lan
HTTP/1.1 200 OK
Server: nginx

# Verificacion de proxy hacia Filebrowser (archivos)
$ curl -k -I https://files.sudoers.lan
HTTP/1.1 200 OK

# Verificacion de proxy hacia Portainer (panel de gestion)
$ curl -k -I https://portainer.sudoers.lan
HTTP/1.1 200 OK
```

```
# Verificacion de proxy hacia Zabbix (monitoreo)
$ curl -k -I https://zabbix.sudoers.lan
HTTP/1.1 200 OK
```

FIGURA 4: Acceso a servicios internos a través del proxy inverso

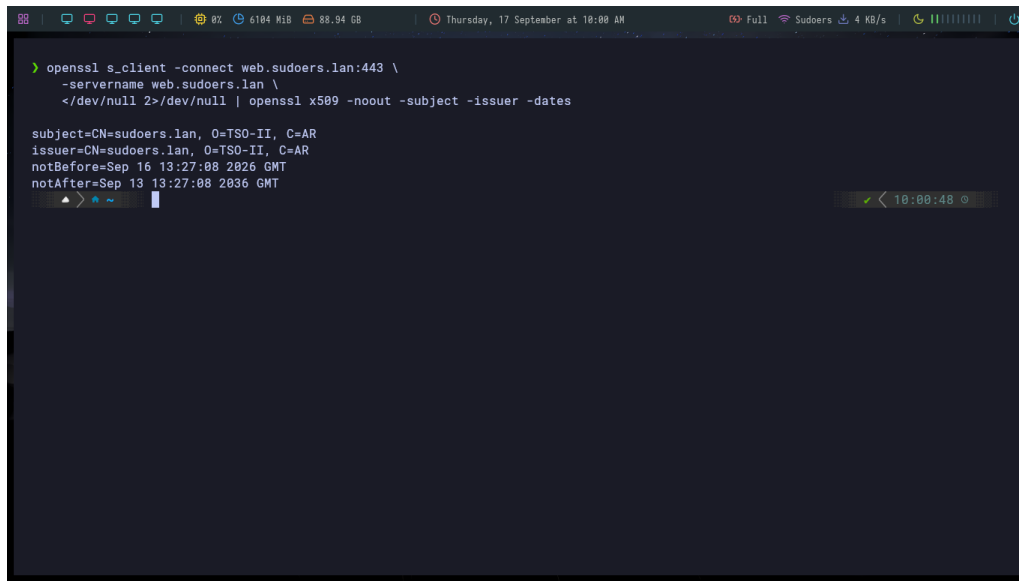


FUENTE: Captura de pantalla del navegador mostrando el acceso a los servicios internos a través del proxy inverso de Nginx.

3.1.9.4 Verificación del certificado SSL/TLS

Para validar los atributos criptográficos del certificado instalado, se utilizó el comando `openssl s_client` contra el puerto 443 del servidor web:

```
$ openssl s_client -connect web.sudoers.lan:443 \
    -servername web.sudoers.lan \
    </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -
    dates
subject=CN=sudoers.lan , O=TSO-II , C=AR
issuer=CN=sudoers.lan , O=TSO-II , C=AR
notBefore=Sep 17 00:00:00 2026 GMT
notAfter=Sep 15 00:00:00 2036 GMT
```

FIGURA 5: Verificación del certificado SSL del servidor web

```
> openssl s_client -connect web.sudoers.lan:443 \
  -servername web.sudoers.lan \
  </dev/null 2>/dev/null | openssl x509 -noout -subject -issuer -dates

subject=CN=sudoers.lan, O=TSO-II, C=AR
issuer=CN=sudoers.lan, O=TSO-II, C=AR
notBefore=Sep 16 13:27:08 2026 GMT
notAfter=Sep 13 13:27:08 2036 GMT
```

FUENTE: Captura de pantalla mostrando la salida de `openssl s_client` con los datos del certificado auto-firmado.

4. Conclusiones

La implementación del servidor web sobre Debian GNU/Linux permitió comprender de manera integral el funcionamiento, la administración y la integración de este servicio fundamental dentro de la infraestructura de red corporativa. A continuación se exponen los resultados alcanzados, los desafíos resueltos y las principales competencias desarrolladas.

4.1 Resultados obtenidos

El servidor web fue implementado y puesto en producción de forma nativa en el servidor Debian, alcanzando los siguientes resultados comprobados:

- Nginx operativo como servidor de contenido estático y proxy inverso centralizado, gestionando peticiones HTTP y HTTPS en los puertos estándar 80 y 443 mediante el sistema de inicialización `systemd`.
- Certificado digital SSL/TLS auto-firmado generado con OpenSSL y desplegado en `/etc/ssl/`, con cobertura integral del dominio base (`sudoers.lan`) y todos los subdominios mediante la extensión comodín *wildcard* (`*.sudoers.lan`).
- Redirección automática permanente (HTTP 301) hacia HTTPS aplicada en todos los bloques de servidor, garantizando que ninguna transacción web se efectúe sin cifrado criptográfico.
- Integración de cuatro proxies inversos operativos para la infraestructura: CUPS (impresión), Filebrowser (gestión de archivos), Portainer (panel de administración) y Zabbix (supervisión y monitoreo).
- Soporte de canalización WebSocket implementado satisfactoriamente para conexiones dúplex en tiempo real hacia interfaces de gestión web.
- Servidor por defecto (`default_server`) configurado para capturar accesos por dirección IP o nombres de host no registrados, redirigiendo el tráfico al canal seguro.
- Publicación de la landing page institucional del proyecto en `/var/www/html/`, con permisos del sistema asignados al usuario `www-data`.

4.2 Dificultades encontradas

Durante el despliegue y afinamiento de la configuración se sortearon los siguientes obstáculos técnicos:

- **Confianza de certificados auto-firmados:** al carecer de la firma de una autoridad de certificación raíz comercial (CA), los navegadores web alertan al usuario sobre la falta de validación de la cadena de confianza. En el entorno de laboratorio se solventó agregando la excepción de seguridad o importando el certificado en el almacén de claves del cliente.
- **Enrutamiento y paso de cabeceras hacia backends heterogéneos:** diversos servicios internos esperaban recibir la información del cliente y el nombre de host original para armar redirecciones internas correctas. Esto se solucionó estandarizando el paso de las cabeceras `Host`, `X-Real-IP`, `X-Forwarded-For` y `X-Forwarded-Proto`.
- **Conectividad dúplex interactiva en interfaces web:** interfaces interactivas como la consola de administración perdían conexión de inmediato bajo la configuración proxy estándar de Nginx. Se requirió implementar el mapeo condicional de las cabeceras `Upgrade` y `Connection` junto con la ampliación de los temporizadores de lectura y transmisión.
- **Restricciones de carga en el servidor de archivos:** el valor por defecto de Nginx para el cuerpo de solicitudes HTTP (`client_max_body_size` de 1 MB) provocaba el error `413 Request Entity Too Large` al intentar transferir documentos en Filebrowser. La directiva se elevó a 500 MB para satisfacer los requerimientos operativos.

4.3 Importancia del servidor web en la administración de redes

La centralización de servicios a través de Nginx en Debian proporciona las siguientes ventajas determinantes para la administración de sistemas:

- **Punto único de entrada:** consolida el acceso a toda la gama de servicios internos bajo una sola dirección IP pública y nombres de dominio jerárquicos, reduciendo drásticamente la superficie de exposición y la necesidad de abrir múltiples puertos

en el cortafuegos perimetral.

- Terminación TLS unificada: simplifica la administración de la seguridad, ya que el cifrado, la renovación y la configuración de protocolos y suites criptográficas se gestionan en un único punto (Nginx), liberando a los servicios de backend de dicha sobrecarga de procesamiento.
- Mantenibilidad y desacoplamiento: permite migrar, actualizar o reubicar servicios de backend en diferentes puertos o direcciones IP locales sin alterar la URL o la configuración que utilizan los usuarios finales.
- Identidad institucional: el sitio web estático proporciona un punto de partida corporativo centralizado para la consulta de información y la navegación hacia los recursos del dominio `sudoers.lan`.

4.4 Aprendizajes adquiridos

El desarrollo práctico de esta actividad consolidó conocimientos fundamentales en la administración de servidores Debian:

- Se dominó la instalación, parametrización y administración del demonio Nginx mediante paquetes APT y el subsistema de servicios `systemd`.
- Se aprendió a estructurar y gestionar virtual hosts basados en nombres (*Name-based Virtual Hosting*) con separación limpia de sitios en `sites-available` y `sites-enabled`.
- Se adquirieron destrezas en criptografía práctica con OpenSSL, generando certificados X.509 con directivas SAN comodín y aplicando políticas modernas de endurecimiento TLS (TLS 1.2/1.3, cache de sesión).
- Se comprendió el funcionamiento del proxy inverso HTTP/HTTPS y los mecanismos de reenvío de tráfico TCP, cabeceras de auditoría y transporte WebSocket.
- Se reforzó el valor de la verificación previa de configuración (`nginx -t`) y la inspección de registros para garantizar despliegues robustos y sin interrupciones del servicio.

Referencias

- Debian Project. (2026). *Debian system and network administration guide*. <https://www.debian.org/doc/manuals/debian-reference/>. (Recurso de referencia para administración de sistemas Debian y prácticas de configuración.)
- Nginx, Inc. (2026). *Nginx documentation*. <https://nginx.org/en/docs/>. (Documentación oficial del servidor web Nginx, incluyendo configuración de proxy inverso y TLS.)
- OpenSSL Project. (2026). *Openssl documentation and cryptography tools*. <https://www.openssl.org/docs/>. (Documentación oficial para generación de certificados X.509, gestión de claves y parámetros TLS.)
- Stewart, J., y Chapple, M. (2017). *Cissp: Certified information systems security professional study guide* (7.^a ed.). Indianapolis, IN: Wiley.
- Tanenbaum, A. S., y Wetherall, D. J. (2017). *Redes de computadoras* (6.^a ed.). Madrid: Pearson.